

Distributed Banking System with Token Ring Mutual Exclusion

Final Project 1 - Distributed Systems

Four independent ATM nodes, each running in its own terminal on localhost, operate on a single shared bank account with no shared memory. Mutual exclusion is guaranteed exclusively by the Token Ring algorithm: exactly one token circulates along the ring ATM1 -> ATM2 -> ATM3 -> ATM4 -> ATM1, and only the node holding the token can execute a transaction (critical section). The account balance travels inside the token message, so it is always consistent across the system.

1. Execution Instructions

1.1 Language and environment

Programming language: Python (tested on Python 3.14; any version ≥ 3.8 works).

Reference operating system: Linux. No compilation is required: atm.py is interpreted directly.

1.2 Libraries and dependencies

None. The project uses only the Python standard library (socket, threading, json, queue, termios). No installation command is needed.

1.3 Starting the nodes

Open 4 separate terminals (one per node) and run, from the project folder:

```
Terminal 1: python3 atm.py 1
Terminal 2: python3 atm.py 2
Terminal 3: python3 atm.py 3
Terminal 4: python3 atm.py 4
```

Each node listens on 127.0.0.1: node n on port 500n (5001-5004). Nodes can be started in any order: if the successor is not up yet, a node keeps the token and retries the delivery every second, so the token is never lost. ATM1 creates the single token about 2 seconds after its start (initial balance 1000) and injects it into the ring. From that moment the token circulates continuously.

1.4 Node-terminal association

Each terminal shows a fixed status panel titled "ATM n" with the node id, its successor, the last known balance, the pending requests and the token position. Moreover every log line is prefixed by [ATM n] and a timestamp, so it is always possible to tell which node is running in which terminal.

1.5 Interactive commands

Commands are typed in the terminal of the chosen node (case-insensitive):

```
DEPOSIT <amount>    queue a deposit
WITHDRAW <amount>   queue a withdrawal (refused if over the balance)
BALANCE              show the last balance seen by this node
HELP                 show the command list
```

A queued request is executed only when the token reaches that node: this is the mutual exclusion at work.

1.6 Logging

Each node logs, on screen and in the file `logs/atm<n>.log`: token reception, token forwarding, transaction begin/commit/rollback and the updated balance. The logs clearly show the movement of the token and the absence of concurrent transactions: every transaction is enclosed between a "Token received" and a "Token forwarded" line of the same node, and merging the four logs by time shows that no two nodes ever transact at the same time.

1.7 Suggested demo (as in the assignment)

Start the 4 nodes and wait for the token circulation (visible in the panels and in the Token received/forwarded lines), then type:

```
on ATM2:  WITHDRAW 200    ->  balance 800
on ATM3:  DEPOSIT 100     ->  balance 900
on ATM4:  WITHDRAW 500    ->  balance 400
```

Optionally, WITHDRAW 99999 on any node shows a refused transaction (insufficient funds).

2. GitHub repository

Repository: <https://github.com/nicola-palo/atm-node>

The repository contains the full commented source code `atm.py` (comments cover token management, message exchange and critical-section logic), the `README.md` with the same instructions, and the demo video `VIDEO.webm` showing the 4 nodes running in separate terminals, the token circulation and the execution of the transactions.

Sistema bancario distribuito con mutua esclusione a Token Ring

Progetto finale 1 - Sistemi distribuiti

Quattro nodi ATM indipendenti, ciascuno eseguito in un proprio terminale su localhost, operano su un unico conto bancario condiviso senza memoria condivisa. La mutua esclusione è garantita esclusivamente dall'algoritmo Token Ring: esiste esattamente un token che circola lungo l'anello ATM1 -> ATM2 -> ATM3 -> ATM4 -> ATM1, e solo il nodo in possesso del token può eseguire una transazione (sezione critica). Il saldo del conto viaggia dentro il messaggio del token, quindi è sempre coerente in tutto il sistema.

1. Istruzioni di Esecuzione

1.1 Linguaggio e ambiente

Linguaggio di programmazione: Python (testato su Python 3.14; funziona su qualsiasi versione >= 3.8).

Sistema operativo di riferimento: Linux. Nessuna compilazione richiesta: atm.py viene eseguito direttamente dall'interprete.

1.2 Librerie e dipendenze

Nessuna. Il progetto utilizza solo la libreria standard di Python (socket, threading, json, queue, termios). Non serve alcun comando di installazione.

1.3 Avvio dei nodi

Aprire 4 terminali separati (uno per nodo) ed eseguire, dalla cartella del progetto:

```
Terminale 1: python3 atm.py 1
Terminale 2: python3 atm.py 2
Terminale 3: python3 atm.py 3
Terminale 4: python3 atm.py 4
```

Ogni nodo ascolta su 127.0.0.1: il nodo n sulla porta 500n (5001-5004). I nodi possono essere avviati in qualsiasi ordine: se il successore non è ancora attivo, il nodo trattiene il token e ritenta la consegna ogni secondo, così il token non viene mai perso. ATM1 crea l'unico token circa 2 secondi dopo il proprio avvio (saldo iniziale 1000) e lo immette nell'anello. Da quel momento il token circola continuamente.

1.4 Associazione nodo-terminale

Ogni terminale mostra un pannello fisso intitolato "ATM n" con l'identificativo del nodo, il proprio successore, l'ultimo saldo noto, le richieste in coda e la posizione del token. Inoltre ogni riga di log è prefissata da [ATM n] e da un timestamp, quindi è sempre possibile riconoscere quale nodo è in esecuzione in ciascun terminale.

1.5 Comandi interattivi

I comandi si digitano nel terminale del nodo scelto (maiuscole/minuscole indifferenti):

```
DEPOSIT <importo>    mette in coda un deposito
WITHDRAW <importo>   mette in coda un prelievo (rifiutato se supera il saldo)
```

BALANCE mostra l'ultimo saldo visto da questo nodo
HELP mostra l'elenco dei comandi

Una richiesta in coda viene eseguita solo quando il token raggiunge quel nodo: questa è la mutua esclusione all'opera.

1.6 Logging

Ogni nodo registra, su schermo e nel file logs/atm<n>.log: la ricezione del token, l'inoltro del token, l'inizio/fine delle transazioni e il saldo aggiornato. I log mostrano chiaramente il movimento del token e l'assenza di transazioni concorrenti: ogni transazione è racchiusa tra una riga "Token received" e una "Token forwarded" dello stesso nodo e, fondendo i quattro log per orario, si vede che mai due nodi transano contemporaneamente.

1.7 Demo suggerita (come da traccia)

Avviare i 4 nodi e attendere la circolazione del token (visibile nei pannelli e nelle righe Token received/forwarded), quindi digitare:

```
su ATM2:  WITHDRAW 200  ->  saldo 800
su ATM3:  DEPOSIT 100    ->  saldo 900
su ATM4:  WITHDRAW 500   ->  saldo 400
```

Facoltativamente, WITHDRAW 99999 su qualsiasi nodo mostra una transazione rifiutata (fondi insufficienti).

2. Repository GitHub

Repository: <https://github.com/nicola-palo/atm-node>

Il repository contiene il codice sorgente completo e commentato atm.py (i commenti coprono la gestione del token, lo scambio di messaggi e la logica della sezione critica), il README.md con le stesse istruzioni e il video dimostrativo VIDEO.webm che mostra i 4 nodi in esecuzione su terminali separati, la circolazione del token e l'esecuzione delle transazioni.